

Project plan for degree project

Template version 1.6 – June 12, 2020

PA2534: MASTER'S THESIS IN SOFTWARE ENGINEERING

March 14, 2026

Title	Benchmarking AI-Assisted Threat Modeling: A Comparative Evaluation of Classical Tools and LLM-Based Approaches	
Classification	Security Engineering; Software Engineering; Artificial Intelligence	
Student 1	Name	Vamshi Krishna Burri
	E-Mail	vabr24@student.bth.se
	Person nr	20030315-4512
	Program	MSc. Software Engineering
Student 2	Name	Ananth Chowdary Ramineni
	E-Mail	anrb24@student.bth.se
	Person nr	20031225-5375
	Program	MSc. Software Engineering
Supervisor	Name & title	Dr. Oleksandr Adamov (Senior Lecturer)
	E-Mail	oleksandr.adamov@bth.se
	Department	Dept. of Software Engineering
Co-advisor	Name & title	Rickard Erwinson (Industry Advisor)
	E-Mail/ phone	rickard@waibloz.com
	Company / HEI**	Waibloz AB

*2012 ACM Computing Classification System: www.acm.org/about/class/2012

**Co-advisor from industry or a higher education institution (HEI).

1 Introduction

Modern software systems are increasingly built from cloud services, APIs, distributed components, and rapidly evolving application logic. As these systems become more complex, identifying security risks during design becomes harder and more resource-intensive. Threat modeling remains one of the main proactive techniques for reasoning about threats before implementation and deployment, helping teams identify attack surfaces, trust boundaries, and likely misuse paths early in the lifecycle [1].

Despite its importance, threat modeling is often difficult to apply consistently in practice. Existing research shows that many approaches still rely heavily on manual reasoning and tool-specific workflows, which can make the process slow, difficult to scale, and dependent on experienced practitioners [3]. This creates tension between the need for early security analysis and the realities of fast-moving software development.

Recent advances in artificial intelligence have created new possibilities for supporting threat modeling tasks. AI-assisted systems, especially those based on large language models, can analyze system descriptions, generate potential threats, and support parts of the reasoning process.

However, recent comparative evidence suggests that current LLM-based threat elicitation tools still show uneven quality and require careful validation before their outputs can be trusted in realistic engineering settings [4].

To address this gap, this thesis compares AI-assisted threat modeling with classical threat modeling methods using the same structured system scenarios. The evaluation focuses on three qualities that are directly relevant to practical security work: threat coverage, precision, and evidence grounding.

1.1 Existing Research

Threat modeling has long been treated as a foundational security-by-design activity, and more recent work has shown that the tool landscape is broad but uneven. Comparative research on automated threat modeling techniques shows that existing tools differ substantially in terms of assumptions, supported workflows, and output quality, making direct comparison difficult [2].

This challenge becomes more important as teams look for approaches that are both rigorous and usable. Empirical work has shown that the structure of the threat modeling language itself can influence the usefulness of the resulting assessment, which suggests that the quality of the method is shaped not only by the analyst but also by the representation used during analysis [5].

The quality of the input artifacts also matters. Recent work on Data Flow Diagrams shows that such artifacts are not merely documentation aids; they also influence how well threats can be validated and justified against system evidence [6]. This is especially relevant for this thesis because evidence grounding is one of the central evaluation dimensions.

At the same time, research on AI systems has expanded the scope of threat modeling itself. Recent work on guardrails for AI systems argues that threat modeling remains important when reasoning about AI-enabled applications, where risk is shaped not only by software components but also by model behavior, data handling, and downstream actions [7]. Related work has also explored how LLMs can be used within DevSecOps processes to support earlier threat discovery and security reasoning [8].

Several recent works are directly relevant to the AI-assisted side of this thesis. ThreMoLIA focuses on threat modeling for LLM-integrated applications and shows why LLM-enabled systems should not simply be treated as conventional software systems with identical assumptions [9]. sAIfe proposes a lightweight threat modeling approach for machine learning application development, while recent benchmark research on LLM-based threat elicitation shows that AI-assisted approaches are promising but still inconsistent in quality and not yet reliable enough to assume expert-level performance [4, 13]. In parallel, recent survey work on LLM security concerns highlights risks such as prompt injection, adversarial attacks, misuse, and agent-related security issues, reinforcing the need for more explicit security analysis of LLM-based systems [14].

1.2 Research Gap

Although both classical threat modeling and AI-assisted approaches are now represented in the literature, the existing evidence base still has an important limitation. Current studies do not yet provide a sufficiently clear comparison of how these two families of approaches perform when they are applied to the same realistic structured system scenarios under the same evaluation conditions.

This limitation appears in several ways. First, comparative work on automated threat modeling tools provides useful insight into the current tool landscape, but it does not establish a

controlled comparison between classical and AI-assisted threat modeling on identical inputs [2]. Second, recent benchmark research on LLM-based threat elicitation shows that current AI-assisted tools can generate plausible outputs, but also that their performance remains inconsistent and below expert-level expectations in important cases [4]. Third, recent studies on threat modeling notation and Data Flow Diagram-based validation suggest that the quality of a threat modeling result depends not only on how many threats are produced, but also on whether those threats can be justified using the underlying system evidence [5, 6].

As a result, an important practical question remains insufficiently answered: when classical and AI-assisted threat modeling approaches are applied to the same software architecture artifacts, which approach produces more useful results for security analysis, and under what conditions? In particular, it is still unclear whether AI-assisted approaches actually improve threat coverage without introducing too many weakly justified or misleading findings, and whether their outputs are grounded well enough to support practical decision-making.

Beyond providing a comparative evaluation, this thesis contributes a controlled and reproducible benchmarking protocol for threat modeling approaches. This includes standardized scenario construction, a normalized reporting format, and explicitly defined scoring rules for coverage, precision, and evidence grounding. By formalizing these elements, the study aims to support transparent comparison, reduce evaluation ambiguity, and enable partial replication or extension in future work. This moves the contribution beyond single-tool evaluation toward a structured evaluation methodology for threat modeling research.

1.3 Key Terminology

"Threat modeling" refers to a structured process for identifying, analyzing, and prioritizing security threats during system design and development.

AI-Assisted Threat Modeling refers to the use of artificial intelligence, especially large language models, to support threat modeling tasks such as generating candidate threats, identifying attack surfaces, or suggesting mitigations.

Classical Threat Modeling Tools refers to non-AI methods and tools that rely primarily on structured frameworks, explicit system models, and human reasoning.

Evidence Grounding refers to the extent to which an identified threat can be justified by explicit features of the analyzed structured system scenarios, such as components, data flows, or trust boundaries.

Benchmarking in this thesis refers to the systematic comparison of different threat modeling approaches using shared inputs, controlled evaluation conditions, and consistent performance metrics.

1.4 Problem Statement

1.4.1 Overview

Threat modeling is widely recognized as an important security engineering activity because it helps organizations identify and address risks before implementation choices are fixed. However, classical threat modeling still requires substantial manual effort, careful interpretation of structured system scenarios, and often specialist knowledge, which can make it difficult to sustain in fast-moving development contexts [1].

Recent AI-assisted approaches promise to reduce that effort by generating threats and supporting early analysis directly from system descriptions. Even so, their practical effectiveness

remains uncertain, especially when outputs appear plausible but may be incomplete, generic, or weakly grounded in the system evidence [4].

1.4.2 Why This Is a Problem

This uncertainty creates a practical problem for both researchers and practitioners. If AI-assisted approaches produce large numbers of candidate threats but many of those threats are incorrect or poorly justified, the technology may increase review effort rather than reduce it. At the same time, relying only on manual approaches may be too slow or too resource-intensive for organizations operating under tight delivery constraints.

The problem becomes even more significant in AI-enabled systems. When LLM-based applications are part of the system under analysis, issues such as prompt injection, adversarial manipulation, misuse, and unsafe autonomous behavior become relevant to the threat modeling task itself [14].

1.4.3 Limitations of Existing Solutions

Existing solutions often fall into two incomplete categories. Classical tools and methods provide structured reasoning but remain labor-intensive, while AI-assisted systems offer speed and flexibility but still struggle with consistency, depth, and dependable validation [2]. As a result, there is still no broadly adopted evaluation framework that allows these approaches to be compared fairly on shared artifacts using practitioner-relevant criteria.

1.5 Industrial Context and Collaboration

This thesis is conducted in collaboration with Waibloz AB, a Swedish startup developing AI-powered reading assistance software for neurodivergent users and second-language learners. The company develops cloud-based software that processes user content and system interactions in real time. This context is suitable for the study because it reflects the kinds of modern software systems where security concerns, data handling, and fast-moving architectural changes all intersect.

Waibloz AB provides industrial relevance and practitioner feedback. However, the main empirical material for the study will consist of public and open-source software systems with documented vulnerabilities or security issues. These systems will be used to construct representative structured system scenarios for benchmarking. This design reduces confidentiality risks, improves reproducibility, and enables comparison against publicly documented security issues. The industrial supervisor will contribute feedback on practical usefulness, while academic supervision will support methodological rigor. [15]

1.6 Ethical, societal and sustainability aspects

The use of AI in security analysis raises both ethical and operational concerns. One key concern is that large language models can generate outputs that are fluent and convincing even when they are incorrect or weakly supported, which may create false confidence if those outputs are not carefully verified [11]. Broader concerns about the limitations of large language models in high-stakes settings also reinforce the need for cautious and accountable use [12].

To address this, AI-generated outputs in this study will be treated as suggestions to be evaluated rather than authoritative security conclusions. Human review remains a core part of the evaluation process in order to reduce the risk of automation bias.

Because the study primarily uses public and open-source systems and publicly documented security issues, the risk of disclosing confidential architectural information is substantially reduced. No proprietary system architecture, internal threat list, or personal data from Waibloz AB will be included in the thesis. Any practitioner feedback obtained from the company will be reported only at an aggregate level and will not disclose sensitive implementation details. If external AI tools are used, prompts will be designed to avoid unnecessary disclosure of sensitive organizational information.

From a societal perspective, improving threat modeling practices supports the development of safer digital systems. This becomes even more relevant as LLM-enabled applications are increasingly used in products and services that process sensitive or high-impact information, while also introducing additional security concerns that require explicit analysis [14].

Sustainability is also relevant. Inefficient security processes increase rework, maintenance effort, and incident response costs. By clarifying when AI-assisted threat modeling provides real benefits and where manual expertise remains necessary, the study aims to support more effective and sustainable security engineering workflows.

2 Aim and objectives

Aim: The aim of this thesis is to compare AI-assisted and classical threat-modeling approaches under shared evaluation conditions using structured system scenarios constructed from public and open-source software systems with documented vulnerabilities or security issues. The study seeks to determine how the selected approaches differ with respect to threat coverage, precision, and evidence grounding, and to assess their practical usefulness for security analysis.[2, 4]

Objectives:

- a. Define a common threat-reporting structure that allows findings from different approaches to be documented and compared in a consistent way.
- b. Construct a small set of representative system scenarios from public and open-source software systems with documented vulnerabilities or security issues to serve as common inputs for all evaluated approaches.
- c. Apply a predefined comparator set consisting of two classical baselines and two AI-assisted baselines to the same structured system scenarios under the same study protocol.
- d. Establish reviewed reference threat models for each scenario through independent first-pass annotation by both authors, followed by documented adjudication and supervisor review.
- e. Measure and compare the evaluated approaches using predefined metrics for threat coverage, precision, and evidence grounding.
- f. Analyze recurring weaknesses and failure modes in the generated threat reports, including unsupported findings, overly generic threats, missing system-specific threats, and weak evidence links.
- g. Collect limited practitioner feedback on the clarity, actionability, and practical usefulness of the resulting threat reports in order to interpret the empirical results from an applied security-engineering perspective.

3 Research questions

This thesis investigates how AI-assisted threat modeling compares with classical threat modeling approaches when applied to the same structured system scenarios. The evaluation focuses on three key aspects: threat coverage, precision, and evidence grounding. The research is guided by the following research questions.

RQ1: How does the threat coverage of AI-assisted threat modeling compare with classical threat modeling when applied to the same structured system scenarios?

Justification: Threat coverage refers to the proportion of relevant threats that are successfully identified. Evaluating coverage is essential because the primary objective of threat modeling is to discover potential security risks early in system design. This question investigates whether AI-assisted approaches are able to identify a broader set of relevant threats compared with traditional manual threat modeling methods.

RQ2: How does the precision of threats identified by AI-assisted approaches compare with those identified through classical threat modeling methods?

Justification: Even if AI-assisted approaches identify a larger number of threats, their usefulness depends on the accuracy and relevance of those threats. This research question evaluates the correctness of identified threats by analyzing false positives and incorrect threat classifications. The question also addresses concerns that AI-generated outputs may produce plausible but incorrect results.

RQ3: How does the evidence grounding of threats generated by AI-assisted approaches compare with those produced by classical threat modeling methods?

Justification: Evidence grounding refers to the extent to which identified threats can be traced back to specific components, data flows, or system properties within the analyzed system. Evaluating grounding is important because security practitioners must be able to justify why a particular threat exists. This question examines whether AI-assisted approaches provide sufficiently grounded explanations to support practical security analysis. Together, these questions aim not only to compare performance but also to reveal trade-offs between completeness, correctness, and evidential support in threat modeling outputs.

4 Method

This thesis conducts a controlled comparative study in software engineering, following the guidelines defined by Wohlin et al. [15]. The study systematically compares different threat-modeling approaches by applying them to the same set of software scenarios. We chose a comparative study design rather than a descriptive case study because it allows us to quantitatively measure and compare threat coverage, precision, and evidence grounding across multiple tools under identical conditions.

The method is designed to address a problem already identified in prior work: direct comparison between threat-modeling approaches is difficult when tools rely on different assumptions, workflows, and output structures [2]. This is especially relevant for AI-assisted approaches, as prior studies show that LLM-generated threats require careful validation [4]. Therefore, the study uses a fixed protocol, shared structured system scenarios, and a common scoring procedure.

4.1 Research Design

The study is organized into five phases: scenario preparation, piloting, reference-model construction, main evaluation, and analysis.

First, open-source software systems with known vulnerabilities (such as OWASP projects) will be transformed into a common structured system scenario for each case. Second, a pilot run will be conducted on one scenario in order to test whether the scenario format, reporting schema, prompting procedure, and scoring guide are sufficiently clear. Third, a reviewed reference threat model will be constructed for each finalized scenario. Fourth, all selected approaches will be applied to the same structured system scenarios under the same study protocol. Finally, the results will be analyzed quantitatively and qualitatively.

This design was chosen instead of a pure case study because the aim of the thesis is not only to describe one company workflow or one tool in depth. The aim is to compare multiple approaches under shared conditions and to determine how they differ in terms of coverage, precision, and evidence grounding. A controlled comparative design is therefore a better fit for the research questions than a purely descriptive case study. Alternative study designs, such as purely descriptive case studies or tool-specific evaluations, were considered but not selected because they would not allow controlled comparison across multiple approaches under identical conditions. All scenario descriptions, prompt templates, scoring rules, and evaluation decisions will be documented to allow reproducibility and independent verification of the study where confidentiality constraints permit.

4.2 Study Materials and Scenario Construction

To entirely bypass corporate confidentiality limitations, the empirical material for this study consists of open-source projects with publicly documented vulnerabilities. These known vulnerabilities provide a partial reference baseline rather than a complete ground truth, since not all possible threats are documented. To keep the work manageable within one semester while still allowing meaningful comparison, the main evaluation will include three scenarios. The use of three scenarios reflects a deliberate trade-off between breadth and depth. A smaller number of well-defined scenarios allows more careful construction of reference models and consistent application of evaluation criteria. Increasing the number of scenarios would reduce analytical control within the available timeframe. The scenarios will be selected to reflect variation in system functionality, data flows, and trust boundaries, while still being documented well enough to support a fair comparison across the evaluated approaches. A scenario will only be included if the available material is sufficient to describe the main components, relevant assets, key interactions, and scope assumptions in a consistent way. This matters because the quality of a threat-modeling result depends not only on the method being used, but also on the quality and structure of the underlying structured system scenarios [5, 6].

Each structured system scenario will include a short system description, a list of relevant assets and entry points, a simplified architecture or component view, a data-flow representation or equivalent interaction view, and explicit assumptions and scope boundaries. During anonymization, company-specific names and business details will be removed or generalized, while the technical structure needed for threat analysis will be preserved. For example, a scenario may represent a specific subsystem such as authentication, data processing, or API interaction, with clearly defined inputs, outputs, and trust boundaries. This ensures that each scenario is sufficiently concrete for consistent threat analysis.

4.3 Compared Approaches and Tools

The study compares classical and AI-assisted approaches. For the classical baseline, we will perform manual STRIDE-based analysis using the Microsoft Threat Modeling Tool and OWASP Threat Dragon [16, 17]. For the AI-assisted baseline, we will use a ThreMoLIA-based workflow [9]. To mitigate API constraints and privacy concerns, the AI approaches will test both cloud-based LLMs (e.g., ChatGPT) and local, open-weight models (such as Llama 3 8B) running on local hardware. All approaches will be applied to the same finalized structured system scenarios. A pilot study will first be used to test whether the worksheet, prompt template, reporting structure, and coding guide are clear enough to use consistently. After this pilot phase, the protocol will be kept fixed throughout the main study.

For the LLM-based approach, each scenario will be run twice in separate fresh sessions using the same predefined procedure, and the model version and access date will be recorded. This is intended to capture variability in LLM outputs while keeping the evaluation feasible and reducing ad hoc variation within the scope of a one-semester thesis. The prompt templates are intended to reflect realistic usage conditions in which practitioners rely on a stable prompting strategy rather than continuous prompt optimization. The prompt templates will be defined before the main evaluation and kept fixed throughout the study. No prompt refinement will be performed after the pilot phase, ensuring that all runs are conducted under identical prompting conditions. If no public ThreMoLIA implementation is available, the published method will be followed as closely as possible, and all interpretation steps will be documented explicitly as a limitation of the study. Recent benchmark work also shows that LLM-based threat elicitation still requires careful validation [4].

4.4 Reference Threat Models

For each scenario, the study will construct a reviewed reference threat model that serves as the evaluation baseline. This baseline is not treated as objective ground truth but as a structured reference point for comparing the outputs of the selected approaches. To reduce subjectivity, both authors will first identify candidate threats independently using the same scenario material and coding guide. Differences will then be resolved through discussion based on the available evidence in the structured system scenarios and the written scoring rules. Supervisors or industry representatives may be consulted only when a scenario itself is unclear; they will not decide the scoring outcomes. This process is intended to reduce the risk that the reference model reflects only one interpretation.

Each reference threat will be defined as a distinct security issue affecting a specific system element or interaction. A threat is considered distinct if it involves a different attack vector, target component, or security property. This definition is used consistently across all scenarios to ensure that the reference model and evaluated outputs operate at the same level of granularity.

Each reference threat will include a short justification linked to explicit elements of the structured system scenarios, such as components, interfaces, data flows, trust boundaries, assets, or stated assumptions. This matters because the reference model should not only identify plausible threats but also demonstrate their support in the available system evidence [6]. To improve reliability, a subset of the finalized reference threats and scored findings will be checked independently after the coding guide has been finalized. Any important disagreements will be documented and discussed in the analysis.

The construction of the reference threat models follows a structured procedure. The process begins with independent identification of candidate threats by both authors. The resulting threat sets are then compared and consolidated through documented discussion. Finally, each agreed

threat is linked to explicit evidence in the structured system scenarios. This stepwise procedure is intended to improve transparency and reduce uncontrolled interpretation.

To further reduce bias, the reference threat models will not be treated as absolute ground truth but as a structured baseline for comparison. The evaluation focuses on relative differences between approaches rather than absolute correctness. In addition, a subset of reference threats will be cross-checked against established threat categories (e.g., STRIDE classifications) to ensure that no major threat classes are systematically omitted. To clarify the role of the reference model, it is not treated as an objective or complete representation of all possible threats. Instead, it functions as a consensus-based baseline constructed under explicit assumptions and documented procedures. The purpose of the reference model is to enable consistent comparison across approaches rather than to establish absolute correctness. As a result, evaluation outcomes should be interpreted as relative differences between approaches within the defined study context.

To further strengthen validity, the reference threat models will be informed not only by independent analysis but also by known vulnerabilities and documented security issues in the selected systems. This reduces the risk of systematically missing critical threats and improves the reliability of the evaluation baseline.

4.5 Evaluation Procedure

All findings and reference threats are treated as comparable units of analysis, defined as distinct security issues linked to specific system elements or interactions. Before scoring, all outputs will be rewritten into a shared threat-reporting format so that the different approaches can be compared on the same basis. Each finding will be recorded as one distinct security issue linked to a system element or interaction. Repeated findings about the same issue in the same context will only be counted once. If one statement contains several separate issues, it will be split before scoring. This normalization step is necessary because the compared approaches produce different kinds of output, and the results must therefore be documented in a consistent format before they can be compared fairly [2].

Each finding will then be checked against the reviewed reference threat model using explicit matching rules.

An exact match is recorded only when: (i) the same underlying security issue is identified, (ii) the relevant system element or interaction is correctly specified, and (iii) the described attack mechanism is consistent with the reference threat.

A partial match is recorded when: (i) the same underlying issue is identified, but (ii) either the system context is incomplete or the attack description is overly general.

A no-match classification is assigned when: (i) the finding does not correspond to any reference threat, or (ii) the statement is too generic to be linked to a specific system element.

All classifications will be supported by short written justifications to ensure traceability and consistency. Threat coverage will be calculated as the share of reference threats identified through exact or partial matches. Precision will be calculated as the share of reported findings that correspond to a reference threat.

Evidence grounding will be operationalized using three observable criteria: (i) explicit reference to a system element (e.g., component, interface, or data flow), (ii) a clear explanation of how the threat arises from that element, and (iii) consistency with the scenario description.

A finding will be classified as grounded only if all three criteria are satisfied. If one or more criteria are missing but partial support is visible, the finding will be classified as partially grounded. Otherwise, the finding will be treated as ungrounded [6].

Because recent benchmark research shows that LLM-based threat elicitation still requires careful validation, AI-generated outputs will be treated as candidate findings rather than authoritative conclusions [4]. To improve consistency, the scoring procedure will be guided by a predefined coding guide that includes representative examples for exact, partial, and no-match classifications. This helps to reduce subjectivity and ensure consistent application of evaluation criteria across all scenarios. A subset of the evaluated findings will be independently scored by both authors using the finalized coding guide. Observed disagreements will be analyzed to identify potential ambiguities in the matching and grounding criteria.

4.6 Analysis

The analysis combines descriptive quantitative comparison with qualitative review.

For each scenario and each approach, the study will report the number of findings, the number of exact matches, the number of partial matches, the number of no-match findings, and the distribution of grounding categories. Results will be summarized per scenario and across scenarios using tables and simple descriptive statistics such as counts, proportions, means, and medians.

Because the number of scenarios is limited within a one-semester thesis, the study will not rely heavily on strong claims of statistical significance. The main emphasis will instead be on transparent scenario-level comparison and pattern identification. This makes the analysis easier to defend and more appropriate for the actual scale of the study.

In addition, a qualitative error analysis will be carried out to identify recurring failure modes, such as overly generic threats, duplicated findings, missing trust-boundary reasoning, weak evidence links, or findings that sound plausible but are not supported by the scenario artifacts. Practitioner feedback will be collected using a short structured questionnaire focusing on three aspects: clarity, perceived usefulness, and actionability of the threat reports. Each dimension will be rated using a simple ordinal scale, and qualitative comments will be collected to support interpretation. This feedback will not be used as primary evidence but as contextual support for interpreting the comparative results.

5 Expected outcomes

This thesis is expected to produce an empirical comparison between classical and AI-assisted threat-modeling approaches using shared structured system scenarios derived from public and open-source software systems with documented vulnerabilities or security issues. Rather than proposing a broad reusable framework, the study is expected to deliver a documented evaluation setup that enables a controlled and transparent comparison within the scope of this thesis.

The expected outcomes of the study include:

- A curated set of anonymized structured system scenarios together with reviewed reference threat models used as the evaluation baseline.
- Comparative empirical results showing how the selected approaches differ with respect to threat coverage, precision, and evidence grounding.
- A structured analysis of recurring weaknesses and failure modes, such as overly generic findings, unsupported claims, missing system-specific threats, or weak evidence links.
- Practitioner-oriented observations on the clarity, actionability, and practical usefulness of the generated threat reports.

- A documented comparison protocol that can support future replication or adaptation where confidentiality constraints allow.

These outcomes are expected to improve the current situation by providing a clearer basis for judging when AI-assisted threat modeling is useful, where it remains unreliable, and where manual expertise is still necessary. The results should be relevant to security practitioners, software teams, and researchers interested in threat-modeling quality and the practical limits of AI-assisted security analysis.

6 Time and activity plan

The thesis work is planned from early January to early June 2026 and follows the official course milestones. The schedule reflects the full project progression, including the initial proposal work completed before the first project-plan submission on February 8, the revision phase after feedback, the benchmark design and evaluation work, and the final thesis deliverables. To keep the study feasible within one semester, the project is limited to a fixed comparator set, a restricted number of scenarios, and a predefined evaluation procedure.

Progress will be monitored through weekly meetings with the academic supervisor and regular coordination with the industry supervisor. Buffer time is included around major milestones to allow for feedback, revisions, and possible delays in tool setup, scenario preparation, annotation, or analysis.

Weeks	Planned Activities and Deliverables
Week 1–4	Topic exploration, supervisor discussions, literature review, research problem definition, and initial methodological considerations.
Week 5–6	Preparation and submission of Project Plan (PP1), including introduction, research questions, method outline, and initial references.
Week 7–10	Revision of proposal based on feedback, refinement of research gap, improvement of literature base, and finalization of method design and benchmark scope.
Week 11–12	Finalization and submission of revised Project Plan (PP2) on March 22.
Week 13–16	Setup of benchmark framework, preparation of structured system scenarios, pilot testing, construction of reference threat models, and development of coding guide.
Week 17–18	Execution of main evaluation, collection of results, and initial analysis of coverage, precision, and evidence grounding.
Week 19–20	Preparation and submission of Thesis Draft (TD1) on May 10, followed by continued analysis and revision of results and discussion.
Week 21–22	Preparation of Opposition Report (OR1), thesis presentation (TP1), and incorporation of feedback.
Week 23	Final proofreading, formatting, and submission of Final Thesis (FT1) on June 7.

Table 1. Week-based timeline from early January to early June 2026.

6.1 Supervision plan

The thesis progress will be monitored through regular supervision throughout the project period. Weekly meetings will be held with the academic supervisor to review progress, discuss

challenges, and plan upcoming research activities. In addition, bi-weekly coordination meetings will be conducted with the industry supervisor to ensure that the research remains aligned with realistic industrial scenarios.

Each supervision meeting will typically address:

- Progress since the previous meeting
- Current results and emerging issues
- Risks and mitigation actions
- Planned activities for the upcoming week

The feasibility of the plan depends on maintaining this restricted scope throughout the project. If delays occur, priority will be given to completing the benchmark design, three finalized scenarios, the reference threat models, and the main comparative evaluation. Extensions such as additional scenarios, broader practitioner review, or any other nonessential analysis will be omitted rather than allowed to delay thesis completion.

7 Limitations and risk management

This section outlines potential limitations and risks associated with the study, together with mitigation strategies designed to reduce their impact.

- **Confidentiality constraints limiting data access.** Industrial partners typically cannot share internal architectures or vulnerability lists due to security protocols. *Mitigation:* The study entirely avoids proprietary architecture. Instead, it relies exclusively on open-source, intentionally vulnerable projects to provide transparent, accessible, and verifiable system artifacts.
- **Incomplete or uneven input artifacts.** Threat modeling quality depends partly on the quality and completeness of the available architecture artifacts. Recent work also suggests that the usefulness of DFD-based validation depends on what information is actually available to the analyst [6]. *Mitigation:* The study will explicitly document which artifacts are provided for each scenario and interpret results accordingly.
- **Weakly grounded AI outputs.** AI-assisted approaches may generate threats that sound plausible but are generic, unsupported, or poorly connected to the actual system context [4]. *Mitigation:* All findings will be evaluated for evidence grounding, and unsupported results will be classified separately.
- **Annotation workload and subjectivity.** Manual validation of threat findings may introduce bias and require substantial effort. *Mitigation:* To improve reliability, a subset of the finalized reference threats and scored findings will be checked independently after the coding guide has been finalized. Any important disagreements will be documented and discussed in the analysis.
- **Tool configuration bias.** AI-assisted results may depend on prompts, model versions, or tool settings. *Mitigation:* Experimental configurations and prompts will be documented explicitly, and pilot runs will be conducted before final evaluation.
- **External validity limitations.** The results may depend on the selected scenarios and architectural domain. *Mitigation:* The study scope will be clearly defined, and limitations of generalization will be explicitly discussed.

A further limitation is the one-semester duration of the thesis project, which restricts the number of scenarios and tools that can be evaluated. To address this constraint, representative

cases and approaches will be selected carefully so that the study still captures a meaningful range of threat modeling situations.

8 Threats to Validity

Several validity risks remain even with a controlled study design. One risk is that the reference threat models may still reflect analyst judgment, since the same authors are involved in constructing the evaluation baseline. This is reduced through independent first-pass annotation, written coding rules, and documented adjudication, but it cannot be removed entirely. Another risk is that LLM-based approaches may produce different outputs at different times, even when the same prompt is used. To reduce this, the study will use fixed prompt templates, fresh sessions, and documented model versions, although a single-run design cannot capture all possible variation. In addition, the conversion of outputs into a common reporting format may introduce interpretation bias, and the limited number of scenarios reduces the strength of generalization. For this reason, the study should be understood as a bounded empirical comparison rather than a universal ranking of threat-modeling methods.

Construct Validity. Construct validity concerns whether the chosen metrics accurately represent threat modeling effectiveness. Metrics such as coverage and precision may not fully capture practical usefulness. To reduce this risk, the study combines multiple evaluation criteria, including evidence grounding and practitioner feedback.

Internal Validity. Internal validity concerns whether observed differences are caused by the evaluated methods rather than unrelated factors such as analyst learning effects or inconsistent procedures. These risks are mitigated by applying each approach to identical structured system scenarios and standardizing the evaluation process.

External Validity. External validity concerns the generalizability of the results. Because the study focuses on cloud-based and web-oriented software architectures, the findings may not directly transfer to domains such as embedded systems or safety-critical cyber-physical systems.

Conclusion Validity. Conclusion validity concerns the reliability of the analytical conclusions drawn from the data. Since the number of scenarios is limited within a one-semester project, statistical uncertainty is possible. Appropriate statistical methods will be used where relevant, and results will be interpreted cautiously. Full reproducibility may be limited due to differences in interpretation during threat identification and scoring, as well as potential changes in LLM behavior over time.

9 GenAI Usage Disclosure

Generative AI tools were used in a limited supporting role during the preparation of this thesis proposal. Their use was restricted to planning support, literature discovery, reference checking, clarity review, and language-level revision of text drafted by the authors. GenAI tools were not used to make final decisions regarding the research problem, research questions, methodology, or interpretation of results. Any AI-supported suggestions were reviewed critically and either revised or rejected by the authors. All academic decisions, analysis, and final wording were determined by the authors.

All referenced sources were verified against original publications, and no AI-generated content was used as a cited source. The authors remain fully responsible for the accuracy, integrity, and originality of the submitted work.

In accordance with the course GenAI policy, a complete record of GenAI use relevant to this proposal, including prompts and output summaries, is provided in Appendix 10.

9.1 Tools used

- **ChatGPT (OpenAI):** Used in a limited supporting role for discussing the already selected topic, identifying candidate literature, and reviewing the clarity of drafted sections.
- **Grammarly and similar language tools:** Used only for grammar, spelling, punctuation, and sentence-flow refinement of text written by the authors.
- **Scopus AI:** Used for initial literature discovery. All papers were manually reviewed and verified by the authors before inclusion.

9.2 Use cases and boundaries

GenAI tools were used only for supporting activities such as planning support, literature discovery, reference checking, clarity review, and language-level revision. AI outputs were treated as suggestions and were critically evaluated before use.

The following tasks were performed entirely by the authors:

- interpretation of results and analytical reasoning,
- verification of all references against original sources,
- final writing and approval of all submitted content.

No AI system was used as a source of scientific content or cited as a reference in this thesis proposal.

9.3 Data protection and confidentiality

No confidential company data, proprietary structured system scenarios, or personal data were included in any GenAI interactions. All prompts used abstracted or anonymized descriptions of systems and scenarios to ensure compliance with data protection requirements.

References

- [1] M. Raciti and G. Bella, “The SPADA methodology for threat modelling,” *International Journal of Information Security*, vol. 24, no. 2, p. 86, Mar. 2025, <https://doi.org/10.1007/s10207-025-00999-0>.
- [2] D. Granata and M. Rak, “Systematic analysis of automated threat modelling techniques: Comparison of open-source tools,” *Software Quality Journal*, vol. 32, no. 1, pp. 125–161, Mar. 2024, <https://doi.org/10.1007/s11219-023-09634-4>.
- [3] Z. Shi, K. Graffi, D. Starobinski, and N. Matyunin, “Threat Modeling Tools: A Taxonomy,” *IEEE Security & Privacy*, vol. 20, no. 4, pp. 29–39, 2022, <https://doi.org/10.1109/MSEC.2021.3125229>.
- [4] D. Van Landuyt et al., “A comparative benchmark study of LLM-based threat elicitation tools,” *Future Generation Computer Systems*, vol. 177, p. 108243, Apr. 2026, <https://doi.org/10.1016/j.future.2025.108243>.

- [5] S. Katsikeas, E. Rencelj Ling, P. Johnsson, and M. Ekstedt, “Empirical evaluation of a threat modeling language as a cybersecurity assessment tool,” *Computers & Security*, vol. 140, p. 103743, May 2024, <https://doi.org/10.1016/j.cose.2024.103743>.
- [6] W. B. Mbaka, X. Zhang, Y. Wang, T. Li, F. Massacci, and K. Tuma, “Assessing the usefulness of Data Flow Diagrams for validating security threats,” *Computers & Security*, vol. 156, p. 104498, Sep. 2025, <https://doi.org/10.1016/j.cose.2025.104498>.
- [7] J. Dev, N. B. Akhuseyinoglu, G. Kayas, B. Rashidi, and V. Garg, “Building Guardrails in AI Systems with Threat Modeling,” *Digit. Gov.: Res. Pract.*, vol. 6, no. 1, Feb. 2025, <https://doi.org/10.1145/3674845>.
- [8] M. Bedoya, S. Palacios, D. Díaz-López, E. Laverde, and P. Nespoli, “Enhancing DevSecOps practice with Large Language Models and Security Chaos Engineering,” *International Journal of Information Security*, vol. 23, no. 6, pp. 3765–3788, Dec. 2024, <https://doi.org/10.1007/s10207-024-00909-w>.
- [9] F. V. Jedrzejewski, D. Fucci, and O. Adamov, “ThreMoLIA: Threat Modeling of Large Language Model-Integrated Applications,” in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, in EASE ’25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 834–839. <https://doi.org/10.1145/3756681.3757083>.
- [10] R. Scandariato, K. Wuyts, and W. Joosen, “A descriptive study of Microsoft’s threat modeling technique,” *Requirements Engineering*, vol. 20, no. 2, pp. 163–180, Jun. 2015, <https://doi.org/10.1007/s00766-013-0195-2>.
- [11] Z. Ji et al., “Survey of Hallucination in Natural Language Generation,” *ACM Comput. Surv.*, vol. 55, no. 12, Mar. 2023, <https://doi.org/10.1145/3571730>.
- [12] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?,” in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, in FAccT ’21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 610–623. <https://doi.org/10.1145/3442188.3445922>.
- [13] G. E. Messas, R. S. Miani, and B. B. Zarpelão, “sAIfé: Towards a Lightweight Threat Modeling Approach to Support Machine Learning Application Development,” in *Proceedings of the XXIII Brazilian Symposium on Software Quality*, in SBQS ’24. New York, NY, USA: Association for Computing Machinery, 2024, pp. 1–10. <https://doi.org/10.1145/3701625.3701640>.
- [14] M. Q. Li and B. C. M. Fung, “Security concerns for Large Language Models: A survey,” *Journal of Information Security and Applications*, vol. 95, p. 104284, Dec. 2025, <https://doi.org/10.1016/j.jisa.2025.104284>.
- [15] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012. <https://doi.org/10.1007/978-3-642-29044-2>.
- [16] Microsoft, “Microsoft Threat Modeling Tool,” *Microsoft Learn*, <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool>.
- [17] OWASP Foundation, “OWASP Threat Dragon,” *OWASP Foundation*. <https://owasp.org/www-project-threat-dragon/>.

10 Appendix A: GenAI Prompt and Output Log

This appendix provides representative GenAI interactions used during the preparation of this proposal. Prompts and outputs are included in their original form with minor formatting adjustments for readability.

10.1 Structure Review (ChatGPT)

Prompt:

Evaluate whether the current proposal structure is logically organized and whether any sections are redundant or unclear.

Generated Output:

The overall structure of the proposal is logically organized and follows a clear academic flow from introduction to methodology. The separation between the research gap and problem statement is appropriate, although there is some overlap in how the importance of threat modeling is discussed in both sections.

You may consider tightening the transition between the research gap and problem statement to avoid repetition. In particular, the explanation of practical challenges in threat modeling appears in both sections and could be consolidated.

The method section is well-structured, but some subsections (e.g., evaluation procedure and reference model construction) could benefit from clearer separation of concepts and definitions.

Overall, the structure is coherent, but minor refinements could improve clarity and reduce redundancy.

Use: Used as feedback on an already written draft. No content was copied directly.

10.2 Literature Discovery (Scopus AI)

Prompt:

Find recent papers comparing automated or AI-assisted threat modeling approaches.

Generated Output:

Several recent studies are relevant to automated and AI-assisted threat modeling:

- Van Landuyt et al. (2026): A comparative benchmark study of LLM-based threat elicitation tools, focusing on coverage and reliability.
- Granata and Rak (2024): Systematic analysis of automated threat modeling techniques with a comparison of open-source tools.
- Jedrzejewski et al. (2025): ThreMoLIA, a threat modeling approach specifically designed for LLM-integrated applications.
- Messas et al. (2024): sAIfe, a lightweight approach for threat modeling in machine learning systems.

These works highlight both the potential and limitations of AI-assisted approaches, particularly in terms of consistency and validation requirements.

Use: Used to identify candidate papers. All papers were manually reviewed and verified before inclusion.

10.3 Language Refinement (Grammarly)

Prompt:

Grammar and spelling suggestions within drafted text.

Generated Output:

Suggestions included corrections to sentence structure, punctuation, and word choice to improve clarity and readability. Some sentences were flagged as overly long or repetitive and were recommended for simplification.

Use: Used only for proofreading and surface-level corrections.

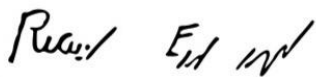
Signature enclosure

The company/organization Waibloz AB accepts the description of the work in this document ("Project plan"). Waibloz AB also accepts to offer the student/students a reasonable amount of supervision in connection with the future degree project in relation to the parts of the degree project that are related to the work at Waibloz AB. Waibloz AB is aware that BTH is not part of any agreement with the company and does not guarantee that the work is carried out in a satisfactory manner.

The part of the degree project that is carried out by the student/students on behalf of Waibloz AB is only a commitment between the student/students and the company. BTH will evaluate the work in the form of documents that are submitted for evaluation. All submitted works can be made public on request. According to regulations, the passed final report will be published in the document management system DiVA and will then become a generally accessible public document.

This signature enclosure is not a legal agreement and is therefore not binding for any party (BTH nor Waibloz AB) but should be considered as a declaration of intention from Waibloz AB.

Karlskrona, March 22, 2026



Signature by supervisor at Waibloz AB

Rickard Erwinson

Printed name